Spatial Data Management

Copyright © 2012 by Morgan & Claypool

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopy, recording, or any other except for brief quotations in printed reviews, without the prior permission of the publisher.

Spatial Data Management

Nikos Mamoulis

www.morganclaypool.com

ISBN: 9781608458325 paperback
ISBN: 9781608458332 ebook

DOI 10.2200/S00394ED1V01Y201111DTM021

A Publication in the Morgan & Claypool Publishers series
SYNTHESIS LECTURES ON DATA MANAGEMENT

Lecture #21
Series Editor: M. Tamer Özsu, *University of Waterloo*
Series ISSN
Synthesis Lectures on Data Management
Print 2153-5418 Electronic 2153-5426

Spatial Data Management

Nikos Mamoulis
University of Hong Kong

SYNTHESIS LECTURES ON DATA MANAGEMENT #21

MORGAN & CLAYPOOL PUBLISHERS

CHAPTER 1

Introduction

The volume of spatial data available for processing increases rapidly over the years with the evolution of sensing devices and telecommunication technology. In addition, the digitization of geographic information is providing an opportunity for common users to routinely issue location-based requests. Most data that can be stored and analyzed carry spatial information; as a result, the management of location and geometric features of entities is an essential component of a modern Database Management System (DBMS). The mapping of many data management tasks to spatial management problems and the maturity of the developed indexing and searching approaches for spatial data has rendered spatial data management a core database research area. Spatial Database Management Systems (SDBMSs) manage large collections of spatial objects, which apart from conventional features include spatial characteristics, such as geometric position and extent. The management of such data types is particularly challenging because the mature relational database technology is not readily applicable.

As an example, consider the set of restaurants in a city. A restaurant has spatial and non-spatial attributes. For example, the name of the restaurant or the food type it serves do not carry spatial semantics. On the other hand, the address of the restaurant, although it is non-spatial in its raw form (i.e., an alphanumeric string), is associated with spatial information, i.e., the *location* of the restaurant on the city map. If we store the set in a conventional database, we would be able to answer any query which refers to the non-spatial features of the data objects. For example, we would be able to find the restaurants that serve a particular type of food, or restaurants of large capacity. On the other hand, it would not be possible to run spatial queries on the database. For example, we would not be able to find the *nearest* restaurant to our current location, using relational database operations. An application program that would exhaustively read all restaurant tuples, translate their addresses to coordinates, and iteratively search for the restaurant nearest to a given location would not only be too slow but also tedious to implement. Therefore, the direct support of spatial attributes of entities that are stored in a database and the development of spatial query operators is important. This is the reason why most commercial and research database systems support spatial data nowadays.

A natural question is why such extensions could not be avoided, since there already exist mature Geographic Information Systems (GISs) that support spatial data management. The reason is that the focus of GISs is different compared to that of an SDBMS; the goal of a GIS is to assist the analysis and visualization of geographic data, which is just one class of spatial data. In addition, unlike SDBMSs, typical GISs do not support set operations; further, GIS operations cannot be easily integrated with other database operations (e.g., aggregation or ranking). In other words, a

GIS cannot replace the functionality of a database system, which has a much more general focus. On the other hand, applications with GIS functionality can easily be built on top of an SDBMS.

In the rest of this chapter, we first provide a brief introduction to spatial data types, predicates, and queries. Then, we discuss the necessary extensions that should be performed to a DBMS in order to effectively support spatial data management. Finally, we discuss the historical evolution of SDBMSs and other applications that use spatial data management technology.

1.1 SPATIAL DATA TYPES, PREDICATES, AND QUERIES

The most commonly supported and frequently used spatial data type is location. Location can be expressed with the help of a coordinate system. It can also directly be modeled and stored in a relation schema. In addition to location, spatial objects often have a *geometric extent*. For example, the extent of a restaurant corresponds to its building area on the city map. Storing the exact extent of an object may increase storage complexity and query processing time. Therefore, in most practical applications, the extent is approximated by a simple geometric shape (e.g., a polygon or a polyline). For example, roads are typically represented as sequences of line segments (i.e., a polyline). This representation is often referred to as *vector approximation*, because a vector data structure is used to implement it in a computer. Still, representing and managing object extents adds to the complexity of the database. For example, a polygon may have a variable number of edges; therefore, a fixed-length data type cannot be used for its representation. In addition, evaluating spatial query predicates (e.g., overlap) over complex object extents can be computationally expensive.

Typically, for location-based queries, point representations or coarse approximations of spatial objects are sufficient. Figure 1.1 illustrates the location q of a mobile user, who is interested in the nearest restaurant. Clearly, due to the sparse distribution of the restaurants (i.e., r_1, r_2, r_3) on the map[1], their extent does not affect the query result and they can simply be represented as points. Storing extents and involving them in query evaluation is essential for applications where query results are affected by extents. For example, in GIS applications, storing spatial details of objects is essential (e.g., for map overlay operations which analyze topological relationships between different layers, such as hydrology, roads, etc.)

In most applications, objects are represented by the vector model (i.e., by simple geometric objects, such as points or polygons). In some applications, however, we often have small collections of voluminous objects with great complexity. For example, consider a meteorological map, which defines regions based on recorded temperatures. There are regions with temperature less than 0 °C, regions between 0 °C and 10 °C, etc. These regions are typically too large and complex to be represented by a simple geometric object. In this case, it is more appropriate to use a *field* representation. A typical model, in this class, is the *raster* approximation, where each object is modeled as a *set* of granules (e.g., pixels). Other field representations include triangulation models, the quadtree decomposition, etc. Figure 1.2 illustrates a vector and a field approximation of a geometric object. In this book, we

[1]The map is a snapshot of a San Francisco neighborhood from Google Maps. The restaurant locations indicated on the map are imaginary.

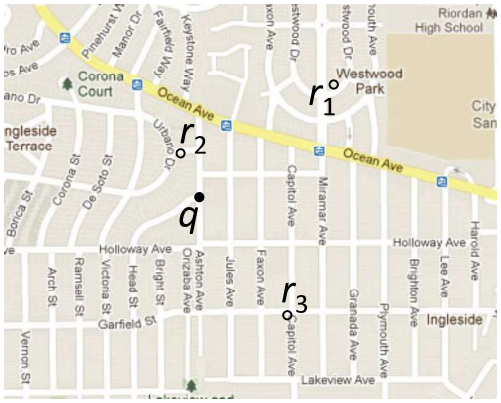


Figure 1.1: Example of a location-based query.

will not consider field representations, which are often used in scientific applications and GIS, but less frequently in SDBMSs.

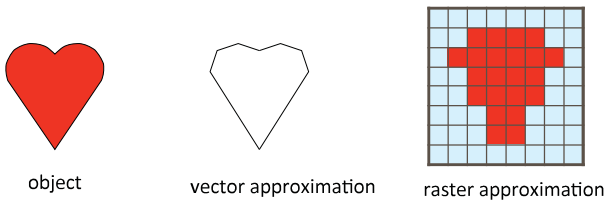


Figure 1.2: Vector and raster approximations of a spatial object.

The spatial data types (i.e., location and extent) that can be associated with entities in a spatial database define a new class of *spatial relationships*[2] between such objects. These relationships characterize the relative location and/or geometry between two objects. They can be classified as topological, directional, and distance relationships. Topological relationships (e.g., overlap, inside, contains, disjoint, etc.) model relationships between the geometric extents of objects. In a natural language, for example, when people say "my school is in the park", they imply that the extent of the school is enclosed by the extent of the park. This is an example of a topological relationship. Directional relationships (e.g., north/south, east/west, above/below, left/right, etc.) compare the relative locations of the objects with respect to a coordinate (or cardinal) system. Combined models

[2]Spatial relationships are often called spatial relations in the literature. Here, we use relationships, in order to distinguish them from database spatial relations (i.e., tables where at least one attribute is of spatial type). Spatial relationships are not to be confused with relationships between entity sets in the Entity-Relationship model; the latter model general associations that may exist between entities.

for topological and directional relationships exist. Figure 1.3 illustrates some examples. Finally, distance relationships capture distance information between two objects. Distance is the hardest to be abstracted. For example, the interpretation of the claim "I am near the train station", in terms of actual distance to the train station, may be hard even for people who know the claimant. As a result, systems use a reference scale to model distance instead of abstract relationships. Even for topological and cardinal relationships, there are no universally standard models, and the convention may differ from implementation to implementation.

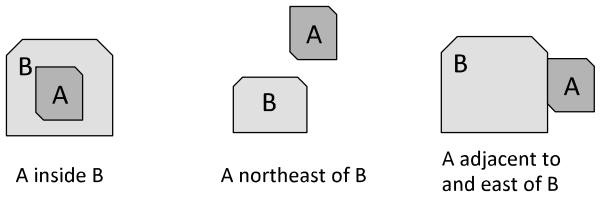
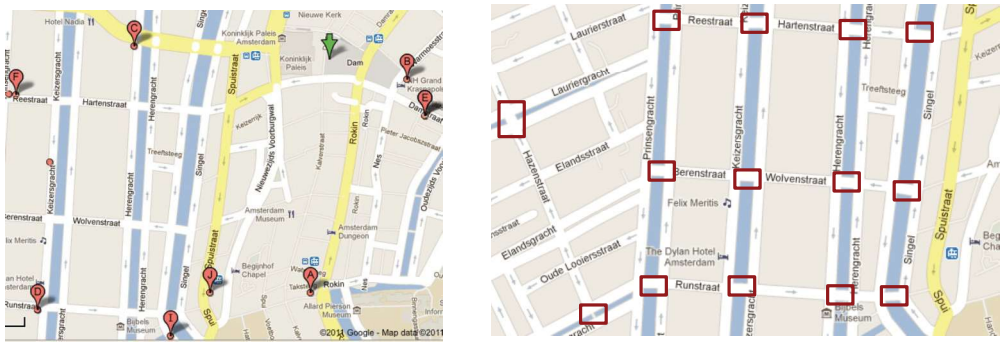


Figure 1.3: Examples of topological and directional relationships.

So, what is the role of spatial relationships in a database system? Like ordinal relationships between numerical data types (e.g., equals, smaller than) that are used in predicates of conventional queries (e.g., find all accounts of balance higher than 1M), spatial relationships are used in predicates of *spatial queries*. For example, the spatial selection query "find all road segments intersecting river Thames" uses the topological relationship "intersects" in its selection condition on a relation that stores the road segments of England. Spatial queries are applied to collections of spatial objects, and use spatial relationships in their predicates. Such predicates are called *spatial* predicates, to differentiate them from conventional query predicates that we find in relational databases. A spatial DBMS supports a set of spatial query operations together with relational operations. The most common spatial query type is the spatial selection, which asks for objects in a relation that satisfy a spatial predicate with a reference object or user-specified region. For example, graphical tools over GISs allow users to define *query windows* and return objects of interest inside these regions. In addition, by typing an address at Google Maps, we implicitly run a spatial selection, which returns points of interest in the vicinity of the location specified by the address.

Additional common spatial query classes are nearest neighbor queries and spatial joins. The former request for the set of closest objects to a reference location. For example, mobile users often use location-based services to browse the closest points of interest to their current location (e.g., nearest gas stations, nearest restaurants, etc.). Figure 1.4(a) illustrates the results of a "nearby pizza restaurants" query from a location at the center of Amsterdam, returned by Google Maps. The spatial joins take as input two collections of objects (e.g., roads and rivers) and a spatial relationship (e.g., intersects) and finds the pairs of objects from the two sets that satisfy the predicate (e.g., pairs of road segments and river segments that cross each other). Figure 1.4(b) shows results of a spatial join that finds intersections between streets and canals in Amsterdam. Additionally, more complex

queries can be defined by injecting ranking and aggregation elements to the basic spatial query types. Query "find the total number of buildings in Central district" is an example of a spatial aggregation query, which extends spatial selection. Query "rank all pairs of hotels and restaurants in the city, in increasing order of their distance, and return the first 50 pairs" extends the spatial join operation, using the distance spatial relationship, with ranking and limiting. Finally, spatial queries often have non-spatial components in them. For example, the "nearest pizza restaurants" query, illustrated in Figure 1.4(a), in fact applies a non-spatial selection (i.e., food type = pizza) before ranking the restaurants with respect to their distance to the query location.



(a) Nearest pizza restaurants to Dam

(b) Street-canal intersections

Figure 1.4: Examples of spatial queries in the city of Amsterdam (maps captured from Google Maps).

1.2 EXTENDING A DBMS TO AN SDBMS

As already discussed, relational database technology is inadequate for managing spatial data. Most database vendors have already responded to the call of extending their systems to include spatial data modeling, language extensions for spatial queries, spatial indexing, and spatial query evaluation techniques.

DBMS technology nowadays supports custom *abstract data types* (ATDs). This extension can readily be used for the definition of the necessary spatial data types. The Open Geospatial Consortium (OGC), an international voluntary consensus standards organization, supports the development and implementation of open standards for spatial data modeling and sharing. The consortium has proposed a specification for spatial ADTs.

The next step is to extend the query language (i.e., SQL) to support spatial queries, based on the defined ADTs. An example of an SQL expression in PostGIS (an open source software program that adds spatial management support to the PostgreSQL object-relational DBMS) is given below.[3]

[3]The example is taken from the PostGIS documentation at http://www.postgis.org/documentation/manual-svn/

```
SELECT
  m.name,
  sum(ST_Length(r.the_geom))/1000 as roads_km
FROM
  bc_roads AS r,
  bc_municipality AS m
WHERE
  ST_Contains(m.the_geom,r.the_geom)
GROUP BY m.name
ORDER BY roads_km;
```

This query takes as input two relations `bc_roads` and `bc_municipality`, which store roads and municipalities, respectively, of British Columbia. The objective of the query is to compute the total length of roads fully contained within each municipality. Each of `bc_roads` and `bc_municipality` contain a spatial attribute `the_geom`. The query spatially joins the two relations: for each municipality, the set of roads contained in it are found (predicate `ST_Contains(m.the_geom,r.the_geom)`). Then, the join results are grouped by municipality name (`m.name`), and for each municipality the lengths of the roads in it are summed. Finally, the municipalities and the lengths are output in decreasing order of their total road lengths as shown below:

```
name                         | roads_km
-----------------------------+------------------
SURREY                       | 1539.47553551242
VANCOUVER                    | 1450.33093486576
LANGLEY DISTRICT             | 833.793392535662
BURNABY                      | 773.769091404338
PRINCE GEORGE                | 694.37554369147
...
```

As illustrated above, the core SQL language semantics are not changed. The language extension includes the introduction of spatial ADTs and a set of new functions that define predicates based on spatial relationships.

Apart from providing means for modeling spatial data and expressing spatial queries, the spatial DBMS should also care about the efficient evaluation of these queries. Spatial query evaluation in a brute force manner can be highly inefficient for two reasons. First, the geometry of the objects could be too complex; therefore, testing a query predicate against each object in a database would result in a high computational cost. Second, exhaustively testing all objects of the relation against a spatial query predicate requires a significant amount of I/O operations, for large databases.

In a spatial DBMS, the first issue is handled by storing, together with the exact geometry of each object, a cheap *spatial approximation*, which can be used as a fast filter. The most commonly used approximation is the *minimum bounding rectangle* (MBR); the MBR of an object is the minimum

rectangle which *encloses* the geometric extent of the object. First, the query predicate is tested against the MBR of the object; if the MBR passes the *filter step*, then the exact geometry of the object is tested against the query predicate (*refinement step*).

The second issue of avoiding checking all objects (even their approximations) can be handled by indexing. Spatial indexing (and spatial query evaluation in general) evolved from data structures that support efficient multi-dimensional range search algorithms in Computational Geometry. The dimensionality and extent of spatial objects does not allow for the definition of an index with theoretical guarantees in its search cost (like the B–tree in relational databases). Still, multi-dimensional indexes that work very well in practice exist, especially for low-dimensional spaces, i.e., the 2D or 3D spatial domain. The most dominant spatial access method is the R–tree. The R–tree defines a hierarchical partitioning of the spatial domain by grouping nearby objects into disk blocks and using as search keys the MBRs of the objects and groups thereof. The hierarchical structure of the index guides search and prunes sub-trees (and the corresponding objects indexed in them) that do not satisfy the query predicate. Other popular indexing methods include grid-based space decomposition and B–tree indexing after transformation to 1D space, using space-filling curves.

The introduction of spatial search operations and spatial indexes into a DBMS increases the evaluation options for complex queries that may involve spatial and non-spatial components. As a result, cost and selectivity estimation models for spatial query components are used in combination with those of relational operations to upgrade the DBMS query optimizer. The query optimizer now has to select among a richer set of potential evaluation plans for a given query.

1.3 HISTORICAL EVOLUTION OF RESEARCH AND SYSTEMS DEVELOPMENT

As in most technology fields, in spatial data management, research precedes development. In the 1980's, spatial data management started as an extension to the existing relational database technology to support the more complex data types found in geographical information systems. In this first decade, the research focus was on appropriate index methods for spatial data, given the inadequacy of relational indexes, like the B–tree. This led to the development of a wide range of indexes. In the 1990's, with the emergence of the object-oriented (OO) model, the focus shifted on spatial database modeling by either directly using OO or extending the relational model. The latter efforts led to the definition of the object-relational (OR) model, which is now used by most DBMSs. In addition, the community dealt with the efficient processing of spatial queries, like nearest neighbor queries or spatial joins and the first indexes for data on spatial (road) networks were developed. In the 2000's, research was aligned with the recent trends and application demands: spatio-temporal data management, continuous evaluation of spatial queries on streaming data, data management for spatial data warehousing and mining, geographic information retrieval, to name a few.

In the 1970's, a number of GIS-like systems that dealt with automated mapping and facilities management were developed; their goal was to digitize maps of city infrastructure (e.g., pipes or transmission lines). In addition, early GISs were developed for the management of geographic

data (e.g., hydrology). All these systems were standalone and stored data directly on file systems. Since 1981, ESRI has led the development of commercial GIS, with the series of ArcInfo (now integrated into the ArcGIS system). ArcInfo, now in its 10th version, is a full-fledged GIS that supports both field and vector spatial data models. Other known GISs include Mapinfo (since 1986), GE Smallworld GIS (since 1990), and the open-source GRASS GIS (since 1997). Major DBMS vendors have extended their products to handle spatial data. Since 1995, Informix (later acquired by IBM) includes spatial data support and an R–tree index implementation. Oracle included basic spatial data capabilities as early as 1984. When Oracle 8 was released in 1997, it included the Oracle Spatial extension, with mature spatial indexing and search support. IBM DB2 includes a Spatial Extender since the late 1990's, which supports spatial data types, spatial predicates, and grid-based spatial indexing. In its 2008 release, Microsoft SQL Server provided spatial data management support, with a choice on indexing (multi-layer grids, space-filling curves and B–trees). The Boeing Company's Spatial Query Server (since 2006) is a commercially available product which enables a Sybase database to contain spatial features. Popular open-source database products, developed in the 2000's, followed a similar path (e.g., PostGIS in PostgreSQL, MySQL, SpatiaLite in SQLite), showing that the support of location and geometry types is an essential element in any DBMS.

1.4 SUMMARY AND OUTLINE

Spatial data management is an essential component of a modern DBMS, due to the fact that many data carry spatial semantics and there are many contemporary applications that search and analyze data spatially. Although Geographic Information Systems are capable of managing large-scale data, they cannot replace an SDBMS, due to their different scope and design.

Spatial objects are characterized by a location and/or a geometric extent. Different approximation levels for the geometry of objects are possible, depending on the demands of the application. The relative location and geometry of objects define spatial relationships between them, which can be used in predicates of spatial queries. The most basic spatial query types are spatial selection, nearest-neighbor search, and spatial joins. Extending a DBMS to support spatial data requires changes at all layers: data modeling, query languages, storage and indexing, query evaluation and optimization, transaction management, etc.

The goal of this book is to provide background on spatial data management issues and techniques to students, researchers, and practitioners. The focus is not on spatial data modeling or query language support for spatial data. Instead, we describe in detail the technology used by the majority of systems in indexing and querying large collections of spatial data objects. Although most of the book can be read by audience with a general background in computer science, it would be more appropriate for readers to be knowledgeable on introductory concepts on database management, including database design, the relational model, query languages, storage and indexing.

Chapter 2 describes the special features of spatial data and explains why relational database systems cannot effectively manage such information. We also overview three classes of spatial relationships that can be used as condition predicates in spatial queries and describe the most common

spatial queries that apply on object collections. Chapter 3 provides an introduction on spatial data indexing and briefly outlines the key issues of this problem. After reviewing some early indexing efforts and discussing their drawbacks, we describe in detail the R–tree, a powerful index for spatial data. Issues like dynamic construction and maintenance of R–trees, as well as bulk loading R–trees for a static collection of spatial objects are covered. In Chapter 4, we show how the most common spatial query types can be processed in a Spatial Database System, including spatial selections, nearest neighbor queries, and spatial joins. Chapter 5 is an introduction on the management of data located on spatial (road) networks. We discuss how the replacement of Euclidean distance by the shortest path distance affects indexing and query evaluation. Finally, Chapter 6 overviews recent developments and trends on spatial data management, including management of spatio-temporal data, similarity search in high-dimensional spaces, top-k and skyline queries, spatial data mining, and spatial keyword search.

BIBLIOGRAPHIC NOTES

There are several textbooks devoted to spatial data management. The textbook by Shekhar and Chawla [2003], based on an earlier survey by Shekhar et al. [1999], is a comprehensive coverage of spatial databases technology. An earlier textbook by Rigaux et al. [2001] focuses on GIS applications. In their book on Object-Relational DBMSs, Stonebraker et al. [1998] discuss the limitations of relational DBMSs to handle complex data types such as spatial. Laurini and Thompson [1992] cover spatial data models, while Worboys and Duckham [2004] discuss implementation issues in systems that manage geographic information. The website of the Open Geospatial Consortium (`http://www.opengeospatial.org/`) includes up-to-date standards on geospatial data modeling and query languages. Zeiler [1999] presents the concepts and techniques used by the ArcInfo GIS for the design and implementation of geographic databases. Besides, Güting [1994] offers an excellent introduction to spatial databases.

Besides the commercial and open-source systems, which we reviewed in Section 1.3, that support spatial data management, there are also several research prototypes, like Paradise [Team, 1995] and SECONDO [Güting et al., 2005].

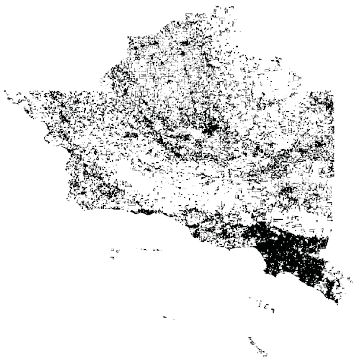
Conference series dedicated to research on spatial data management include the annual ACM SIGSPATIAL International Conferences on Advances in Geographic Information Systems (ACM GIS), the biannual Symposia on Spatial and Temporal Databases (SSTD), and the annual IEEE International Conferences on Mobile Data Management (MDM). GeoInformatica is a journal, published by Springer, which covers spatial modeling and databases.

CHAPTER 2

Spatial Data

An object is characterized as *spatial* if it has at least one attribute that captures its location in a 2D or 3D space. Moreover a spatial object is likely to have geometric *extent* in space. For example, we can say that a building is a spatial object, since it has a location and a geometric extent in a 2D or 3D map. As another example, in a large-scale map, we can consider cities as spatial objects with location, but no extent (i.e., we can model them as points).

A collection of spatial objects which have the same semantics is a *spatial relation*. More formally, a spatial relation is a table, where each row corresponds to a spatial object and each column to an attribute of spatial or non-spatial type. Figure 2.1 shows an example of a spatial objects collection and the corresponding spatial relation. In this example, the spatial attribute of each object is modeled by a polyline. Note that spatial objects may have non-spatial attributes (e.g., name). In this example, we have modeled the spatial attribute of an object as a *vector* of spatial co-ordinates. The approximation of the spatial features of an object using vectors is very popular since it is cheap and can be used at multiple resolutions of the data space.



(a) a collection of road segments

ID	Name	Type	Polyline
1	Boulevard	avenue	(10023,1094), (9034,1567), (9020,1610)
2	Leeds	highway	(4240,5910), (4129,6012), (3813,6129), (3602,6129)
…	…	…	…

(b) a spatial relation

Figure 2.1: Example of a spatial relation.

The need for efficient management of spatial objects emerged from Geographic Information Systems (GISs), which provide mechanisms for the visualization and analysis of geographic data. Digitization of geographic information brought to availability large spatial maps of various thematic contents which need to be efficiently analyzed. Spatial databases not only store geographic content. Spatial objects are found in segmented medical images (i.e., objects in an X-ray), components in

computer-aided design (CAD), constructs or VLSI circuits, stars on the sky, components of molecular structures, objects in microscopic images, etc.

More and more users are interested in retrieving information related to the locations and geometric properties of spatial objects. Users of mobile devices may want to find the nearest hotel to their location. Astrologers may want to study the spatial relationships among objects of the universe. Army commanders may want to schedule the movements of their troops according to the geography of the field. Scientists may want to study the effects of object positions and relationships in a 2D/3D space to some scientific or social fact (e.g., spatial analysis of protein structures, relationship between the residence of subjects and their psychic behavior, etc.).

The rest of this chapter reviews the types of spatial relationships between objects, summarizes the most important types of spatial queries, and introduces the special characteristics of spatial data that complicate their efficient management.

2.1 SPATIAL RELATIONSHIPS

A *spatial relationship* associates two spatial objects according to their relative location and extent in space. People frequently use spatial relationships in their natural language. Consider, for instance, the expression "my house is close to Central Park." In this expression, *close to* is a spatial relationship which implies an upper distance bound between the two objects *my house* and *Central Park*. This is an example of a *distance* relationship. Other important classes of spatial relationships are *topological* and *directional* relationships.

2.1.1 TOPOLOGICAL RELATIONSHIPS

An object is characterized by the space it occupies in the universe, which can be considered as a subset of the set of pixels in the universe. Conceptually an object has a *boundary*, which is defined by the pixels that are adjacent to at least one pixel outside the object, and an *interior*, which is the set of pixels occupied by the object, but are not part of its boundary. Topological relationships associate two objects based on the set-relationships that hold between their boundaries and interiors. Figure 2.2 illustrates a hierarchy of simple topological relationships that may exist between two objects. Observe that the relationship *intersects* implies one of the *equals, inside, contains, adjacent, overlaps*, i.e., it is a generalization of all of these topological relationships. In other words $intersects(o_1,o_2) \Leftrightarrow \neg disjoint(o_1,o_2)$. Table 2.1 shows how these relationships can be defined by logical expressions on the interiors and boundaries of the objects. For example, object o_1 is inside object o_2 (denoted by $inside(o_1,o_2)$) if and only if the interior of o_1 is a subset of the interior of o_2.[1] Specializations of *inside* and *contains* are also possible, if we consider the potential boundary intersection of the associated objects. For example, in the illustration of the *contains* relationship in Figure 2.2 the boundaries of the two objects intersect. This relationship could be differentiated from another *contains* relationship

[1] In fact, the definitions of Table 2.1 assume that both objects have interiors. In the general case, where an object can have only boundary (e.g., points, lines), the definitions are slightly altered. We leave this as an exercise to the reader.

where boundaries are disjoint. In addition, more complex topological relationships can be defined if we consider objects with holes and/or non-contiguous areas.

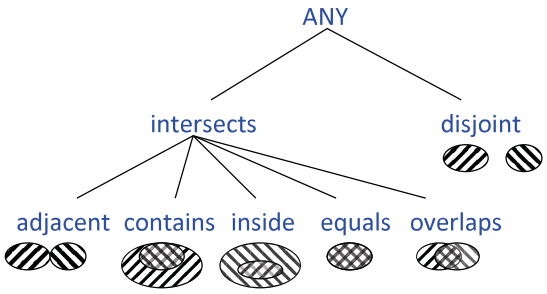


Figure 2.2: A set of simple topological relationships.

Table 2.1: Definitions of topological relationships.

Topological relationship	equivalent boundary/interior relationships
$disjoint(o_1, o_2)$	$(interior(o_1) \cap interior(o_2) = \emptyset) \wedge (boundary(o_1) \cap boundary(o_2) = \emptyset)$
$intersects(o_1, o_2)$	$(interior(o_1) \cap interior(o_2) \neq \emptyset) \vee (boundary(o_1) \cap boundary(o_2) \neq \emptyset)$
$equals(o_1, o_2)$	$(interior(o_1) = interior(o_2)) \wedge (boundary(o_1) = boundary(o_2))$
$inside(o_1, o_2)$	$interior(o_1) \subset interior(o_2)$
$contains(o_1, o_2)$	$interior(o_2) \subset interior(o_1)$
$adjacent(o_1, o_2)$	$(interior(o_1) \cap interior(o_2) = \emptyset) \wedge (boundary(o_1) \cap boundary(o_2) \neq \emptyset)$
$overlaps(o_1, o_2)$	$(interior(o_1) \cap interior(o_2) \neq \emptyset) \wedge (\exists p \in o_1 : p \notin interior(o_2) \wedge p \notin boundary(o_2))$ $\wedge (\exists p \in o_2 : p \notin interior(o_1) \wedge p \notin boundary(o_1))$

2.1.2 DIRECTIONAL RELATIONSHIPS

Directional (or cardinal) relationships associate two objects based on their (relative) orientation with respect to a global reference system. Examples of directional relationships are *north, south, east, west, northeast*, etc. The reference system could also be defined with respect to the orientation of a viewer or a reference object. Examples of such relationships are *left, right, above, below, front, behind*, etc. Directional relationships can be subjective. For example, there may not be a clear border between south-west and west. Therefore, spatial data models often use fuzzy definitions of directions (e.g., 20% north, 80% east).

2.1.3 DISTANCE RELATIONSHIPS

Distance relationships associate two objects based on their distance, which is measured by a (geometric) distance metric, e.g., the Euclidean distance. Actual distance values are not always useful because humans tend to classify them in (subjective or objective) ranges. For example, we can divide the domain of possible distances between objects in a city to distances up to 100 meters, characterized by the relationship *near*, distances from 100 meters to 1km, characterized by the relationship *reachable*,

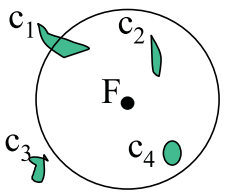
distances from 1km to 10km, characterized by the relationship *far*, and distances larger than 10km, characterized by the relationship *very far*. Therefore, distance relationships can be expressed either explicitly (by the actual geometric distance between the objects) or implicitly (by a distance range).

Topological, distance, and directional relationships can be combined to characterize a pair of spatial objects (e.g., "my house is disjoint with, 100 meters from, northwest of West Bank"). Spatial relationships are used to assist the expression of spatial queries. In the next paragraph we will describe some of the most common spatial query types.

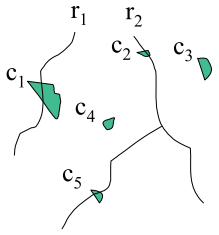
2.2 SPATIAL QUERIES

A spatial query is applied on one (or more) spatial relations and asks for objects (or combinations of objects) that satisfy some spatial relationships with a reference query object (or between them). Spatial queries are the reason for devising specialized management methods for spatial data, in the same way as relational queries determine the way relational data are stored, indexed, and accessed.

The most common spatial query type is the *spatial selection* (or *spatial range query*), which asks for objects in a spatial relation that satisfy a spatial predicate with a well-defined spatial region or object. As an example consider a spatial relation that stores information about cities and the query "find all cities intersected by the Danube river". The response set includes cities whose boundaries or interiors intersect the polyline which represents the Danube river. As another example consider a spatial relation storing cities, as depicted in Figure 2.3a, and the query "find all cities within 100 km distance from the point F". In this case the circle with center F and radius 100 km defines the spatial region of the selection and the response set is $\{c_1, c_2, c_4\}$. The simplest selection query is the point query, i.e., we want to find the objects that contain a given point.



(a) a spatial selection on Cities



(b) spatial join between Cities and Rivers

Figure 2.3: Example of two spatial queries on relations Cities and Rivers.

Another common spatial query type is the *nearest neighbor query*, which, given a well-defined reference object q, asks for the nearest object (or for the k-nearest objects) to q in the spatial relation. The query "find the city which is closest to the point F" is a nearest neighbor query, which, when applied to Figure 2.3a, will retrieve city c_2.

Both selection and nearest-neighbor queries apply to a single relation. The *spatial join* is a query that combines two relations, retrieving the subset of their Cartesian product that qualifies a spatial predicate (i.e., a spatial relationship). Formally, given two spatial relations R and S and a spatial relationship θ, the spatial join $R \bowtie_\theta S$ is defined as $\{(r, s) : r \in R, s \in S, r\ \theta\ s\}$. As an example, consider two relations that store cities and rivers as depicted in Figure 2.3b and the spatial join query "find all pairs of cities and rivers that intersect". The response set of this query is $\{(c_1, r_1), (c_2, r_2), (c_5, r_2)\}$.

Besides the three basic spatial query types that we have discussed, there are also additional, more sophisticated operations on spatial data. For example, so far, we have assumed that the spatial objects are *atomic* entities, i.e., they are not decomposable and they are treated as units in set operations. In GIS applications, however, a spatial object may be decomposable and we could be able to express queries that generate new objects, using partial information from the input data. For example, consider a query, which returns the geometries of rivers that pass through a query region, excluding their parts outside the query region. As another example, consider a definition of the spatial intersection join, which instead of returning pairs of objects that intersect, computes and outputs the common (intersection) regions between these pairs. Objects can also be merged to define new, composite objects (fusion). In addition, spatial aggregation operations can be defined by extending the spatial selection operation (e.g., "what is the number of lakes in a given geographic region?"). Finally, complex queries can be defined by combining the basic spatial query operations, we discussed so far (e.g., "list the 5 nearest service stations to my location; list the restaurants (if any) within 100 meters from each of these stations").

Query languages can easily be extended to support spatial query expressions. For example, selection predicates in the SQL WHERE clause may include functions that compare the geometric extents of objects or measure distance between them. In general, the language extensions are straightforward (Chapter 1 contains an example in PostGIS) and covering them in detail is beyond the scope of this book. On the other hand, as we discuss in the next section, efficient spatial query evaluation is non-trivial, due to the special nature of the data.

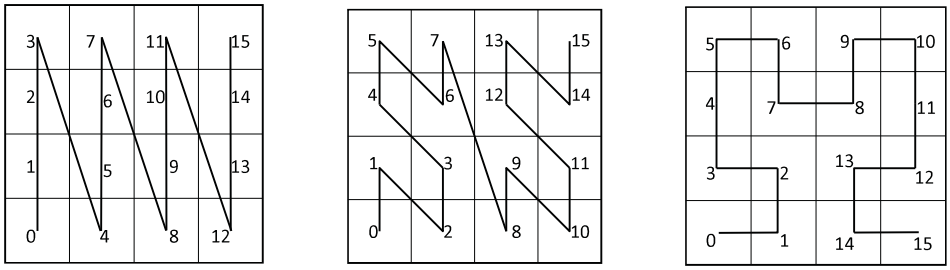
2.3 ISSUES IN SPATIAL QUERY PROCESSING

The spatial features of an object are typically stored physically as a sequence of point coordinates that define a polygonal or polyline approximation of the object. In the simplest case, where objects are points, a single d-tuple per object is stored (d is the dimensionality of space). While objects can be represented in a straightforward way, access methods and query processing techniques for relational databases are not readily applicable for spatial databases due to the following properties of spatial data:

- There is no total ordering in the multidimensional space that preserves spatial proximity. As a result, objects in space cannot be physically clustered to disk pages in a way that provides theoretically optimal performance bounds to spatial queries.

- The spatial extents of objects add to the complexity of physical clustering imposed by dimensionality and do not allow for a standard and closed definition of spatial operators and algebra.

To comprehend the first issue, assume that we try to sort a set of two-dimensional points using some one-dimensional sorting key. No matter which key we use, we cannot guarantee (for arbitrary point sets) that any pair of objects, which are close in space, will also be close in the total order. This can be demonstrated in Figure 2.4 which shows three orderings of pixels (i.e., cells) on a 4×4, two-dimensional map. These orderings are also called *space filling curves*. No matter which curve is chosen at sorting, we can find a pair of objects for which the distance in the ordering does not reflect the actual distance in space. For instance, points at positions 3 and 12, in Figure 2.4b, are relatively close to each other, but they are very far in the linear order defined by the curve. Thus, there is a lack of a linear ordering method in the 2D (and, in general, multi-dimensional) space that preserves spatial locality for arbitrary data distributions. The implication of this is that we cannot achieve good theoretical bounds for the cost of spatial selections (and spatial queries in general) and that we cannot guarantee that this cost linearly depends on the selectivity of the queries. On the other hand, instances of relational data types (i.e., numbers, strings, etc.) can be totally ordered and the worst-case cost of range queries on them is logarithmic to the relation size and linear to the query output size. Although there exist main memory data structures that provide good theoretical bounds on 2-dimensional range queries on points (e.g., the *range tree*), to date, no dynamic, disk-based access method can provide acceptable worst-case bounds for spatial data.



(a) column-ordering (b) Z-ordering or Peano-curve (c) Hilbert-ordering

Figure 2.4: Spatial filling curves.

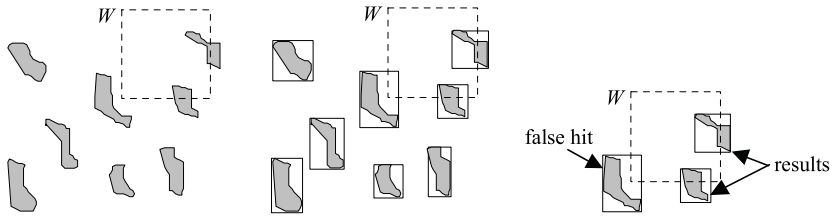
The spatial extent further complicates indexing, since it cannot be modeled by a simple key, even in the 1D space. Consider for instance the problem of indexing 1-dimensional intervals. Sorting the intervals according to one end-point or some other uniquely defined point (e.g., the interval's center) does not optimally solve the problem of finding intervals that intersect a point. Some main memory solutions (e.g., interval tree, segment tree) cannot be straightforwardly transformed

to disk-based indexing methods. More importantly, they cannot be applied to objects of higher dimensionality and more complex extent.

Apart from the indexing problem, the complex geometry of the objects renders the evaluation of spatial predicates inefficient when applied directly on their exact representations. Consider, for instance the set of objects depicted in Figure 2.5a and a query that asks for all objects that intersect the window W. The cost of applying the query directly on the polygonal representations is high, since expensive computational geometry algorithms are required. More specifically, the cost of checking the intersection of two polygons is in the worst case $O(n \log n)$, where n is the total number of edges in the polygons. In order to reduce the large computational overhead of spatial queries, along with each object, a *conservative* approximation is stored. The most common approximation is the *minimum bounding rectangle* (MBR), which is the minimum rectangle that encloses the object.

The spatial query is then processed in two steps. During the *filter* step, the query is applied on the object MBRs. If the MBR of the object does not qualify the query, then the exact geometry does not qualify it either. This can be demonstrated in Figure 2.5b, where MBRs that do not intersect W do not enclose objects that qualify the query. The filter step is a computationally cheap way to prune the search space, but it only provides candidate query results. During the *refinement* step, the exact representations of the candidates are tested with the query predicate to retrieve the actual results. In Figure 2.5c, observe that only two from the three candidates are query results. In some cases the refinement step can be avoided. For example, if at least one side of the MBR is inside W, then the object definitely intersects W.

For most (topological) spatial relationships between two objects, the tightest relationship that should hold between their MBRs is *intersects*. As a result, in many cases, when we are talking about spatial queries, we refer to the filter step (which is essential for pruning large parts of the database), considering the intersection spatial relationship.



(a) objects and a query (b) object MBRs (c) candidates and results

Figure 2.5: Two step query processing using object approximations.

2.3.1 EXTENT OR NOT?

More often than not, in large collections of spatial objects, the extents of the objects are very small compared to the world where they exist. For example, cars and buildings on a city map are very small

compared to the area of the city. The impact of this is that for queries that implicitly or explicitly refer to a large area of the map, the object extents play negligible role to the query result; approximating the objects as points and treating them as a point dataset would have small, if any, effect to the result of the query.

As a result, in many spatial database applications, object extents could be ignored whenever the scale of the query is much larger than the scale of the objects. For example, it is typical for location-based queries (e.g., find the nearest restaurant) to consider the locations of query and the data objects as points. The technical implication of this is that storage, indexing, and search can be greatly simplified in the SDBMS; the expensive refinement step is not necessary for most queries as details about the geometry of objects are disregarded.

2.4 SUMMARY

Spatial data objects are characterized by location and extent. These features define special sets of relationships between them: topological, directional, and distance relationships. Based on these relationships, we can define new predicates in database queries; spatial queries extend the traditional selection, ranking, and join operations to include spatial relationships. The management of spatial data is challenging because their dimensionality and extent complicates indexing and query evaluation. A common technique to alleviate the high cost of spatial query evaluation is to define cheap approximations of the objects and use them as fast filters. The next two chapters elaborate more of spatial indexing and query evaluation.

BIBLIOGRAPHIC NOTES

Our definition for spatial relations is based on the object-relational data model for complex data management [Stonebraker et al., 1998]. Spatial data modeling in a DBMS has been an important issue, since the late 80's [Laurini and Thompson, 1992], with the development of several, mainly object-oriented, models [Borges et al., 2001, de Oliveira et al., 1997] and extending the UML and ER models [Shekhar et al., 1997].

There has been a significant amount of research on the definition and reasoning with topological [Egenhofer, 1991, Egenhofer et al., 1994, Renz and Nebel, 1999], distance [Zimmermann, 1993], and directional [Chen et al., 2010, Freksa, 1992, Ligozat, 1998, Skiadopoulos et al., 2004, 2007] spatial relationships. Papadias et al. [1995] study the conversion of topological relationships between objects to relationships between their corresponding MBRs. They also show how to apply search for such relationships on hierarchical spatial access methods (e.g., R–tree variants) and compare the effectiveness of such methods.

Güting [1994] summarizes the fundamental and advanced [Scholl and Voisard, 1989] spatial operations, typically implemented in database systems that support geographic data. A nice introduction about the special issues in spatial query processing is given by Gaede and Günther [1998].

The filter-and-refine framework has been considered by early efforts to use DBMS technology for the management of spatial data [Frank, 1981, Orenstein and Manola, 1988].

CHAPTER 3

Indexing

The special nature of multidimensional data stimulated the database community to work on the challenging problem of spatial indexing. As a result, a number of *Spatial Access Methods* (SAMs) have been proposed. The role of a SAM is to group the objects into disk pages such that objects in the same page are close to each other in space. The disk pages are then organized into an index; single-level and hierarchical indices have been proposed. This chapter reviews some of the most important efforts on indexing spatial data, focusing on the R–tree, the predominant spatial access method.

3.1 POINT ACCESS METHODS

Many early SAMs are applicable to low-dimensional points, which are easier to manipulate than objects with extent. Most *Point Access Methods* (PAMs) decompose the space into disjoint regions and group points in these regions together into blocks, which are then indexed with the help of a (hierarchical) directory. The main differences between these methods are on how they define the space partitioning and how they maintain it during data updates. In this section, we describe some of the most characteristic indexes in this class.

3.1.1 THE GRID FILE

A classic point access method is the *grid file*. The grid file divides the space into cells, using axis-parallel hyperplanes. Consecutive cells with a large enough number of points in them are grouped into disk blocks (i.e., the points in each group are physically stored in the same block). A *directory* block holds the mapping between cells and disk blocks for easy access. Figure 3.1 is a simple illustration of the grid file. The space is divided into 16 cells, 9 of which contain points. These points are divided into 3 blocks. The directory (not shown explicitly) maps each cell to a block on the disk (block-ids of cells are illustrated by the different colors). The index can be used to answer a window query (i.e., spatial selection) as follows. First, the cells that intersect the query area are identified. Then, using the directory, the blocks that correspond to these cells are found and fetched from the disk. The points in these blocks are compared against the query range and the results are retrieved. The grid file is constructed dynamically, starting from a single cell (and block) that corresponds to the entire space and splitting using hyperplanes, whenever a cell's capacity exceeds the block size and a single block cannot be used to store it. The grid file has similar properties to hash indexes in relational databases. It is good for static data of relatively uniform distribution, since in that case the points are uniformly partitioned into the cells and the disk space is well utilized. However, for

skewed distributions, blocks corresponding to sparse areas may be under-utilized and the directory may become extremely large. In addition, dynamic updates may result in expensive split (or merge) operations, where a large number of cells are affected by a single split.

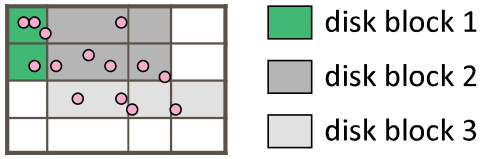


Figure 3.1: A simple grid file.

3.1.2 SPACE FILLING CURVES

Space-filling curves were proposed as early as in 1890, by mathematicians who wanted to create continuous mappings between the one-dimensional and multi-dimensional spaces. This idea turned out to be useful also in spatial indexing. A space-filling curve defines a mapping from a multi-dimensional domain to a one-dimensional domain; that is, every point in space is mapped to a unique number (i.e., distinct points map to different numbers). Therefore, the curve defines a *linear* ordering of the possible points in the multi-dimensional domain. In addition, two points that are close in space are likely to have close mappings, as the curve is a continuous line that fills the entire space.

Figure 3.2 shows the Z-order curve (a.k.a. Peano curve) at three different granularities of the 2D space. The curve is recursively defined as follows. Initially, each dimension is divided into 2^n units ($n = 1$); this division defines four cells, which are ordered by an 'N' curve as shown in the leftmost frame of the figure. At each iteration, the domain of each dimension is doubled; each cell from iteration n is divided into four cells in iteration $n + 1$; the four newly created sub-cells are again ordered by an 'N' curve. The space is divided using as many iterations as necessary, in order to index the multi-dimensional domain at an adequate granularity. In practice, to determine the 1D mapping of a given (x, y) coordinate, the construction of the curve is not necessary. For a given granularity (determined by n), we first construct the n-bit binary representations of x and y. Then, we interleave the bits of x and y, to construct a $2n$-bit number. This number is the Z-order of the point at (x, y). For example, consider the point denoted by the small red circle in the rightmost frame of Figure 3.2. The coordinates of this point are $x = 6$ and $y = 4$, with binary representations $x_2x_1x_0 = 110$ and $y_2y_1y_0 = 100$, respectively. The 6-bit Z-order value of the point is then $x_2y_2x_1y_1x_0y_0 = 111000$, which corresponds to number 56. The Z-order curve is easy to define and nicely preserves spatial locality. Another popular mapping is the Hilbert curve, illustrated in Figure 2.4c.

Using space-filling curves to index spatial data points is relatively simple. A space filling curve is used to define one-dimensional keys for the indexed points. The points are then indexed like relational data with the help of a B^+–tree. The tree can be used to evaluate spatial range queries as

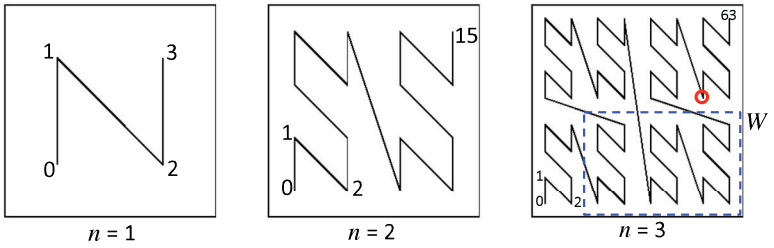


Figure 3.2: Three iterations of the Z-order curve.

follows. Note that the 2D space covered by the query range W contains a set of Z-order addresses. These addresses can be grouped to continuous value intervals on the curve. For example, in the rightmost frame of Figure 3.2, the query range W illustrated by the dashed rectangle contains all Z-order curve values in [8, 15] and [32, 47]. For each interval, we issue a range query to the B^+–tree and retrieve the query results. This way, a spatial range query is transformed to a set of one-dimensional range queries. Because the curve preserves spatial locality, the number of intervals for a random query is expected to be small; therefore, this indexing method is expected to behave well. An additional advantage is that index updates are performed in logarithmic time on the B^+-tree. Space filling curves have been used for indexing spatial data in some systems (e.g., Microsoft SQL Server), and they are mostly appropriate for cases where the spatial data domain and granularity are mostly static.

3.1.3 THE QUADTREE

The quadtree is a tree structure, which is not necessarily balanced, where each non-leaf node has exactly four children. The root node corresponds to the complete 2D space; the children of each non-leaf node divide its area into four equal *quadrants*. Leaf nodes correspond to disk blocks, storing the points that are assigned to the corresponding regions. Space partitioning is done such that each leaf node does not contain more points than the capacity of a disk block. Therefore, the space may be unevenly partitioned by the quadtree; the partitioning follows the data distribution.

Figure 3.3 illustrates a simple quadtree. The left part of the figure shows the distribution of the indexed points and the space partitioning applied by the tree. On the right, the tree structure is shown. The non-leaf nodes are denoted by circles and the leaf nodes by squares. Each leaf node corresponds to a disk page, storing the points that are included in the corresponding partition. In this example, we assume that a disk block can store at most 3 points, so regions with more than 3 points should recursively be repartitioned. The root node has four children corresponding to the top-left, bottom-left, top-right, and bottom-right quadrants of the whole space. Note that the bottom-left and top-right quadrants have at most 3 points, so they do not need to be repartitioned. The points in them are assigned to disk blocks, which are indexed by the quadrants. On the other hand, the top-left and bottom-right quadrants of the root are partitioned recursively.

Spatial range queries are evaluated using the tree structure to find the blocks that overlap with the query area and examining the points in them. The regular partitioning imposed by the quadtree structure allows for smart addressing schemes for the leaf nodes (i.e., disk blocks that contain points). Block addresses can be indexed by a B^+–tree, which can be used to efficiently search for the leaf nodes that overlap with a given query area (similar to the indexing methodology based on space filling curves). One issue with the quadtree is that, due to the regular partitioning it imposes in the whole space and the requirement that each node should have four children, many leaf nodes may be under-utilized (i.e., they may have very few or no children). This issue can be alleviated by techniques that allow multiple leaf nodes to be stored in a single disk block (a method that is also applied for the cells of a grid file).

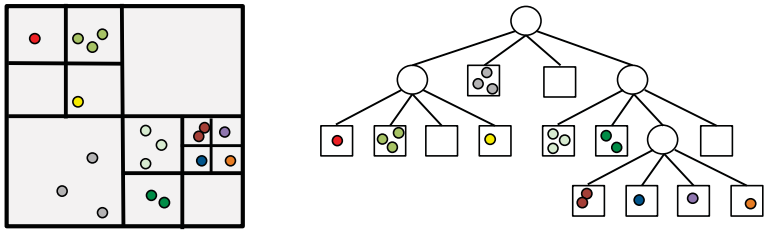


Figure 3.3: Illustration of a quadtree. Left: space partitioning and locations of points. Right: tree structure and distribution of points to leaf nodes.

3.2 INDEXING OBJECTS WITH EXTENT

Point Access Methods (PAMs), some of which we reviewed in Section 3.1, are appropriate for indexing points, not objects with extended geometry. Most PAMs define space decompositions of *disjoint* partitions, which are sufficient for clustering points into disk blocks. However, while a point is guaranteed to be assigned to a unique partition, a spatial object with extended geometry may overlap with more than one partition. For instance, the grid file decomposes the space using an orthogonal grid, each point is assigned to exactly one cell, and adjacent cells are organized to disk pages. The structure cannot directly be used for indexing objects, since grid lines may bisect objects.

One possible way to use PAMs to index spatially extended objects is to apply *object clipping*; an object that is contained in more than one partition is split accordingly and its components are distributed to different disk blocks. For example, consider the set of objects shown in Figure 3.4a. For the sake of indexing, assume that we wish to partition these objects into 3 groups of 4 objects each, such that the minimum rectangular areas that enclose the partitions are pairwise disjoint. Such a task is impossible. Therefore in order to achieve good balance among the partitions, while keeping them pairwise disjoint, the only solution is to clip some objects, as illustrated in Figure 3.4b. This idea can also be applied to indexing based on space-filling curves, where each object is decomposed

to a set of granules, each corresponding to a value of the curve, and eventually represented by a set of value intervals on the curve.

Object clipping leads to data redundancy and replication, due to the possible decomposition applied to objects. If the indexed objects have large extents, data replication may affect the performance of indexing. An alternative way to index spatial objects that avoids replication is by considering not necessarily disjoint regions at space decomposition. Figure 3.4c illustrates an example. In order to achieve good balancing in the partitioning, we allow the two rightmost partitions to intersect each other. This results into three partitions, each having four objects. This idea has been employed by the R–tree, which we describe next.

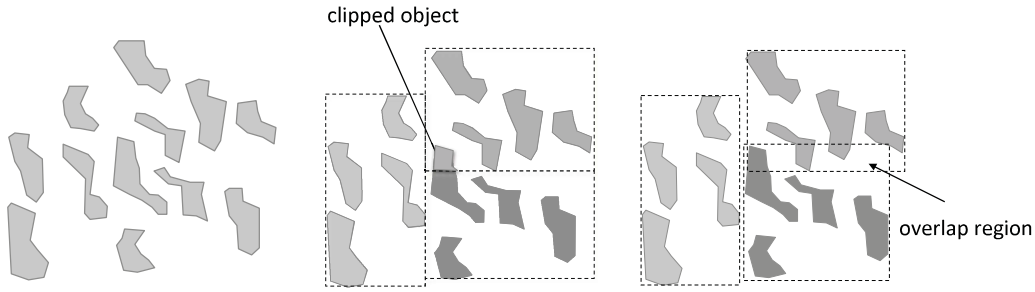


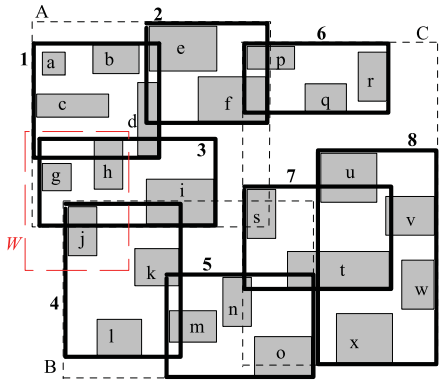
Figure 3.4: Object clipping vs. non-disjoint partitions.

3.3 THE R–TREE

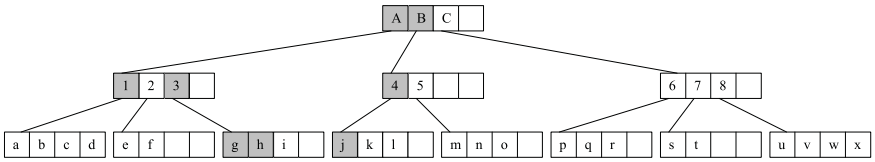
The most popular spatial access method is the R–tree. The R–tree has been implemented in several commercial database products that manage spatial data (e.g., Illustra, Postgress, Oracle, MySQL, Mapinfo). It is a height-balanced tree with structure very similar to the B^+–tree. Like most spatial access methods, the R–tree does not index the exact extent of the objects, but their MBRs. The two-step query processing technique discussed in Chapter 2 is therefore applied; first, the R–tree is used to find fast the object MBRs that qualify the query and then the exact extent of those objects is accessed and verified against the query.

Each R–tree node corresponds to a disk page (i.e., disk block) and consists of entries of the form (MBR, ptr). In leaf node entries, MBR is the minimum bounding rectangle of a data object and ptr is the id of the object. The object-id, equivalent to the record-id in relational database indices, addresses the location of the object in the spatial relation. In non-leaf node entries, MBR is the minimum bounding rectangle of all data objects under the R–tree node pointed by ptr. The number of entries per node is determined by the page size. A constraint of the tree is that each node (except from the root) should be at least 40% full so that disk space is utilized properly. In other words, each node should have between m and M entries, where $m = 0.4M$.

Figure 3.5 shows some object MBRs indexed by an R–tree and a query window W. The query is processed by recursively traversing the nodes (starting from the root) that intersect the window. For example, entry C does not intersect W, thus the subtree pointed by it needs not be examined. The gray R–tree entries in Figure 3.5b intersect W and define the paths that are traversed during search. Notice that although entry **1** intersects W, no object below it is in the response set; intermediate R–tree nodes cannot avoid covering empty space and the overlap between nodes may cause traversing multiple paths even for point queries. However, when a good insertion algorithm that minimizes dead space and overlap between nodes is used, the R–tree performs very well for typical query ranges and data distributions. In this graphical example, the R–tree has a small maximum node capacity M ($M = 4$). In real implementations, the capacity, depending on the disk page size (4Kb-16Kb), can be from a few to several hundreds.



(a) object MBRs (in gray), MBRs of R–tree entries, and a query window W



(b) the R–tree; nodes touched by query W are in Gray color

Figure 3.5: A set of MBRs and an R–tree built for it.

3.3.1 OPTIMIZATION OF THE R–TREE STRUCTURE

The R–tree is a dynamic data structure. This means that updates (i.e., insertions and deletions) interleave with search operations. In order to maintain good performance, the tree employs appropriate algorithms that dynamically organize rectangles in the hierarchical disk structure. There are several

properties of the tree nodes that affect the search performance of this data structure. We outline them below:

- **The *area* covered by the MBRs of the directory node entries should be minimized**. Observe that the MBR of a directory node entry is defined by the MBRs of all objects in the subtree pointed by it. In Figure 3.5, for instance, the entry labeled **1** points to a disk block that contains objects a, b, c and d. The MBR of this entry is the MBR that encloses all these objects. Observe that there is some *dead space* in the MBR of entry **1**. Dead space corresponds to space not occupied by any object MBR under **1**. The (negative) effect of dead space is that the disk block pointed by **1** will be accessed if a query intersects its MBR, even when it does not intersect any object in that disk block. Minimization of the MBRs of all directory node entries implies that the dead space will be minimized and search will be more efficient. For instance, if instead of a, b, c and d, we had grouped a, b, c and x in the same node, the area of the corresponding node and the dead space would be very large and our tree would not be very effective.

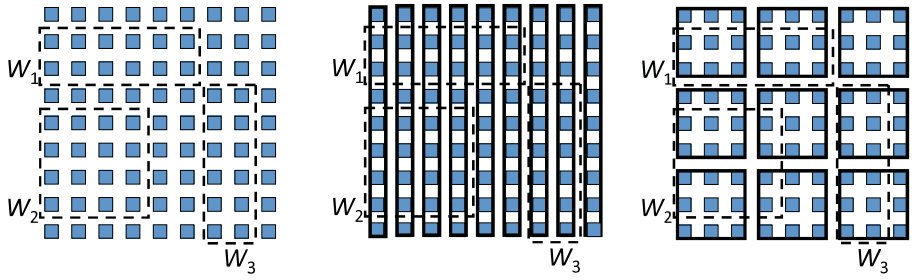
- **The *overlap* between MBRs of the directory entries at the same level should be minimized**. In our running example, consider the root entries A, B, and C. The overlap between these entries should be minimal; otherwise, multiple search paths could be followed by the same query. To see the effect, assume that entry **2** was assigned to node pointed by B (instead of A). In this case, entry B would have large overlap with A and C and many disk nodes would be visited for a window query enclosing the space around entry **2**.

- **The *margins* of the MBRs of the directory node entries should be minimized**. The margin of a rectangle is the sum of the lengths of its four sides. Assuming a fixed area for a rectangle, its margin is minimized if it is a square. In simple words, the shapes of the MBRs at the directory node entries should be as quadratic as possible, instead of being long stripes. The rationale is that the expected number of fixed-area MBRs touched by an arbitrary query is minimized if they are more square-like. To understand the intuition behind this optimization, consider the problem of Figure 3.6. Here, we have a set of 81 objects to be grouped into 9 disk blocks such that the number of blocks touched by an arbitrary query window (such as the dashed W_1, W_2, and W_3) is minimized. A possible grouping is shown in Figure 3.6b. Another grouping is shown in Figure 3.6c. In this case, the margins of the groups are minimized. Note that the grouping of Figure 3.6c minimizes the average number of blocks accessed by the three queries. Only W_3 is favored by the grouping of Figure 3.6b; thus, this grouping it is not appropriate for arbitrary queries.[1]

- **The average number of entries in an R–tree node should be maximized**. In other words, the tree nodes should be as full as possible. Large occupancy of nodes means fewer nodes (and

[1] Observe that the dead space in the grouping of Figure 3.6c is larger compared to the one of Figure 3.6b. This illustrates why grouping of rectangles is a hard optimization problem; optimizing one parameter may negatively affect another optimization criterion. It also shows that margin minimization prevails area minimization. Later, we will see how these metrics can be combined in a good R–tree insertion algorithm.

fewer disk blocks) and shorter search paths from the root to the leaves. On the other hand, packing rectangles to tree nodes allows less freedom for optimal partitioning with respect to area, overlap, and margins minimization.



(a) objects and three queries (b) striped groups (c) square-shaped groups

Figure 3.6: The effect of margins minimization in grouping of MBRs into blocks.

All four optimizations should be considered when updating an R–tree. Assume that we have a (dynamic) set of objects which we want to index. The R–tree can be constructed by iteratively inserting objects into an initially empty tree, consisting of a single (root) block. Once the root becomes full, we need to *split* it into two blocks, considering the optimization criteria. Apart from node splitting, another important optimization is how to choose the most appropriate path when inserting a new object entry in an R–tree.

3.3.2 THE R*–TREE: AN OPTIMIZED VERSION OF THE R–TREE

Several heuristics have been proposed in order to optimize insertion and node splitting in R–trees. Here we will discuss the algorithms employed by the R*–tree, an optimized version of the R–tree. The structure and search algorithms for the R*–tree are identical to those of the R–tree. The only difference between these access methods is the techniques employed for insertion and splitting.

Assume that we have an R–tree structure, where we want to insert a new entry e, containing the MBR of an object and a reference pointer to it. Figure 3.7 describes a recursive procedure that determines the most appropriate leaf node to accommodate e. Function *choose_subtree* takes as input the new entry e and the root node n of the R–tree node and returns the leaf node where e should be inserted. While the current node is not a leaf node, the best entry to be followed is determined based on how its MBR changes after e is accommodated in the subtree under it. For entries pointing to leaf nodes, the minimum increase of overlap is the factor that determines the entry selection. In the case where the overlap increase is the same and minimum for multiple entries, the entry with the least area enlargement is chosen, and if we still have more than one such entries, the entry with the smallest area. For entries pointing to non-leaf nodes, only the area enlargement and the smallest area are considered.

```
function choose_subtree(Node n, Entry e): Node leaf
1.   if n is a leaf then
2.     return n;
3.   if entries of n point to leaf nodes then
4.     fe := entry in n, which will incur the least overlap with
            other entries in n if e is accommodated in the node pointed by it.
            If there are multiple such entries choose the one with
              the minimum MBR enlargement if e is stored under it.
            Break ties by choosing entry with the minimum area;
5.   else //entries of n point to directory nodes
6.     fe := entry in n, with the minimum MBR enlargement if e is stored under it.
            Break ties by choosing entry with the minimum area;
7.   fn := fe.ptr; //node pointed by fe
8.   return choose_subtree(fn, e);
```

Figure 3.7: The *choose_subtree* algorithm of the R*–tree.

After inserting an entry in an R*–tree leaf node, the node may *overflow*, i.e., it may have more entries than the capacity of a disk block. In this case, the node is split into two new nodes and one entry is added to the parent node of the split one. If the parent node overflows too, it is split recursively.

When splitting a node, we partition a set of rectangles into two groups such that the MBRs of these groups have small areas, overlap, and margins. In addition, each group should contain at least m rectangles. A brute-force approach would consider all combinations of possible splits and choose the one with the best quality. However, the cost of such an approach is extremely high. The R*–tree uses the algorithm of Figure 3.8 for node splitting. First, for each dimension i, the projections of the rectangles on dimension i are considered; these projections are value intervals. The rectangles are sorted into two lists; one using their lower bounds in that dimension and one using their upper bounds. For each list, all possible splits to two groups are considered and the sums of the margins of the MBRs for each group pair are summed. Then the axis with the smallest sum of margins in all distributions is chosen as the split dimension. The intuition is that if we split across the "longest" axis, the resulting nodes will be more quadratic. After the split dimension is chosen, the best distribution along that axis is chosen, this time considering minimum overlap between the resulting groups (and minimum area as a second criterion, in the case of ties).

In order to increase the space utilization of nodes (i.e., to increase the average occupancy of nodes by entries), an additional heuristic is used by the R*–tree. Instead of splitting an overflown node immediately, 30% of its entries (which are far from the center of its MBR) are temporarily removed and re-inserted. The intuition is that these entries may enter some other node and overflow could be avoided. If these entries are re-inserted into the same node, the algorithm proceeds with splitting, since it cannot be avoided. This *forced re-insert* significantly improves the tree structure and makes it less sensitive to the order by which rectangles are inserted. On the other hand, this approach may result in multiple tree paths being traversed during an insertion, which complicates the application of concurrency-control.

```
function split(Node n): Node n1, Node n2
1.   for each dimension i
2.       Si := 0;
3.       sort the entries using the lower bounds of the rectangles in dimension i;
4.       for k:=m to M + 1 − m
5.           place first k entries in group A and the remaining ones in group B;
6.           Si := Si + margin(A) + margin(B);
7.       sort the entries using the upper bounds of the rectangles in dimension i;
8.       for k:=m to M + 1 − m
9.           place first k entries in group A and the remaining ones in group B;
10.          Si := Si + margin(A) + margin(B);
11.  split dimension := dimension i with the minimum Si;
12.  sort the entries using the lower value of the rectangles in split dimension;
13.  for k:=m to M + 1 − m
14.      place first k entries in group A and the remaining ones in group B;
15.  sort the entries using the upper value of the rectangles in split dimension;
16.  for k:=m to M + 1 − m
17.      place first k entries in group A and the remaining ones in group B;
18.  from all the above groupings (A, B), choose the one such that
         the overlap between A and B is minimized;
19.  if there are multiple groupings with minimal overlap,
20.    choose from them the one with the smallest area(A) + area(B);
21.  put in n1 all rectangles from A; put in n2 all rectangles from B;
22.  return (n1, n2);
```

Figure 3.8: The *split* algorithm of the R*–tree.

Finally, deletion in an R–tree is handled in a much simpler way compared to the equivalent operation of the B$^+$–tree; if a leaf node underflows after some deletion, all the remaining entries are temporarily deleted and re-inserted into the structure. In this way, the insertion algorithm facilitates optimized re-location of the rectangles and complex processing of deletions is avoided.

3.3.3 BULK-LOADING R–TREES

Given a set of spatial objects, an R–tree can be built for it by iteratively inserting each object into an initially empty tree. However, this method has two disadvantages if all data are available a-priori (static data). First, iteratively invoking the R–tree insertion algorithm is too slow. Second, most of the pages in the resulting tree will have a lot of empty space (typically, the nodes of an R*–tree are 67% full).

A good *bulk-loading* method would build fast, for a static set of objects, an R–tree with maximum node occupancy (and thus minimum height) and high node quality with directory MBRs of minimum area, margin and overlap between them. We can follow the sort-based bulk-loading paradigm, which is successfully used for B$^+$–trees. A brute-force approach would be to choose a sort axis (e.g., the x-axis) and sort the object MBRs (or points) with respect to their projection on this axis (e.g., the center of the x-projection). The R–tree can then be built in a bottom up fashion by